Binary Search Tree

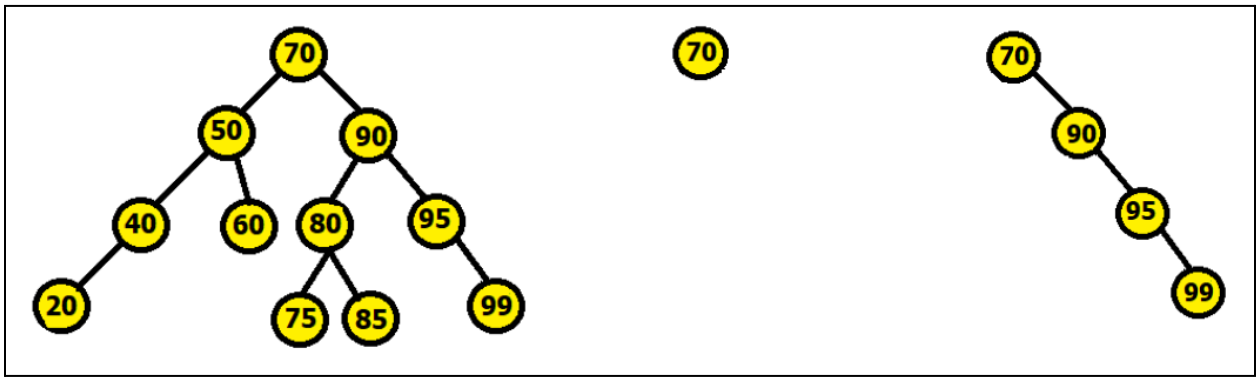
Characteristics of a BST:

Binary Search Tree is a binary tree data with the following fundamental properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtrees must also be binary search trees.
- Each node must have a distinct key, so no duplicate values are allowed.

The goal of using BST data structure is to search any element within O(log(n)) time complexity.

Examples of Binary Search Tree:



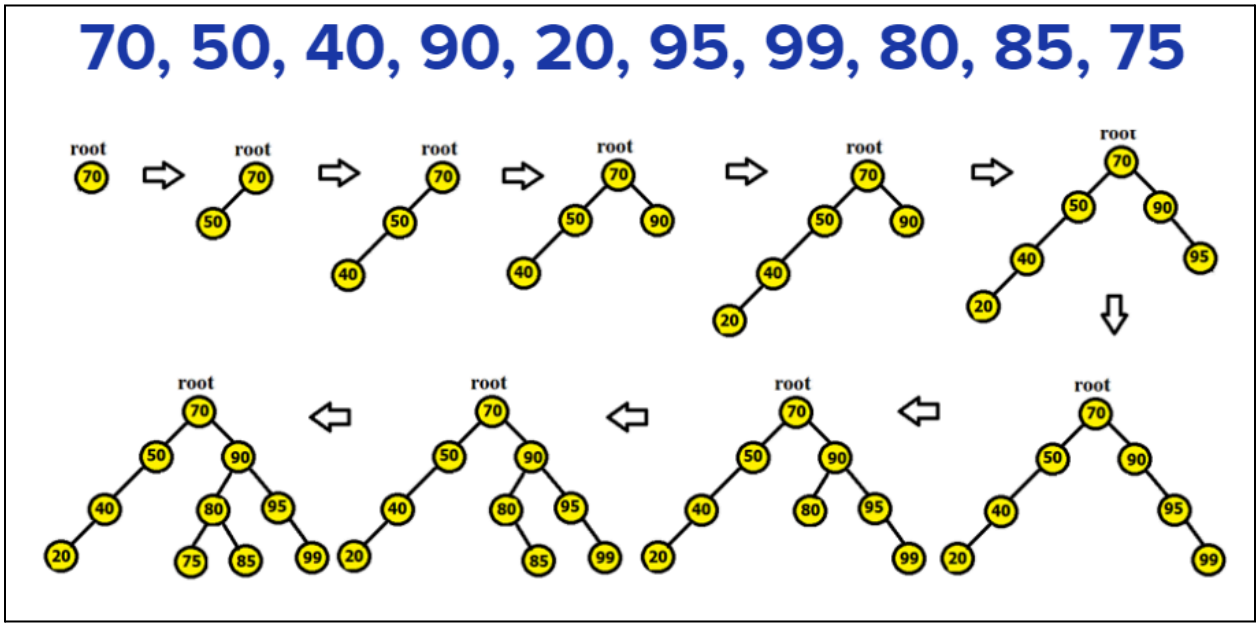
Binary Search Tree Operations:

Any operation done in a BST must not violate its fundamental properties.

- **Creation:**

Create the BST by inserting the following numbers from left to right:
70, 50, 40, 90, 20, 60, 20, 95, 99, 80, 85, 75

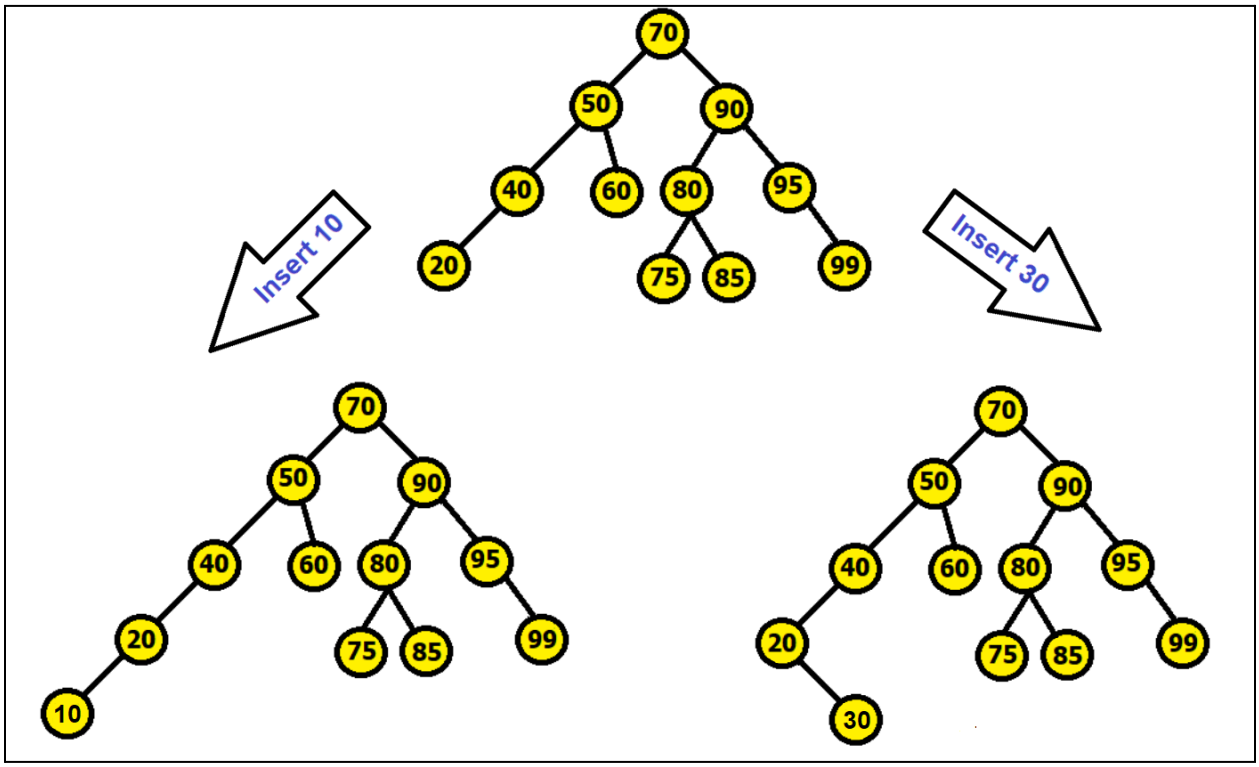
i) Take the value and compare with root.
ii) If the value < root, go to left subtree
iii) Else if the value > root, go to right subtree
iv) Keep going until a vacant space is found and insert the new value



- **Insertion:**

Similar to Creation

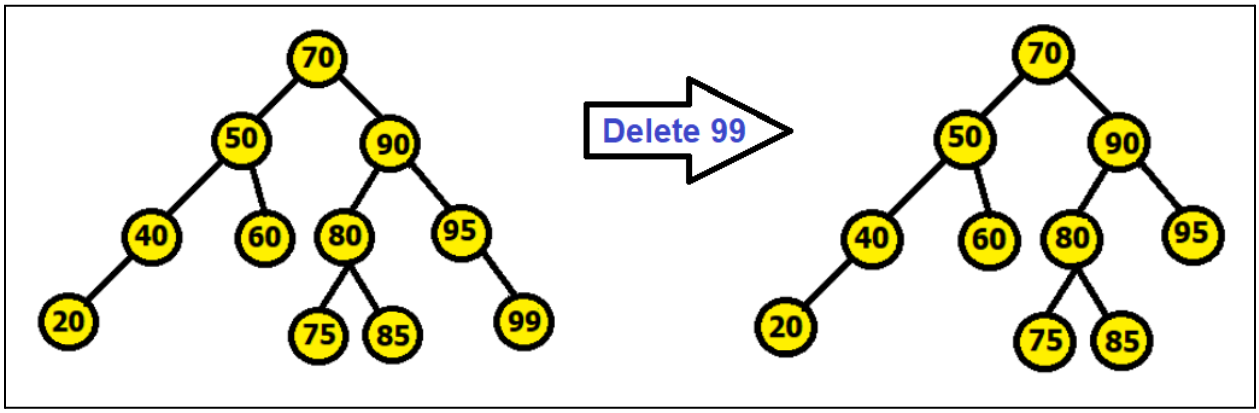
i) Take the node and compare with root.
ii) If node value < root, go to left subtree
iii) Else if node value > root, go to right subtree
iv) Keep going until a vacant space is found and insert the new node



- **Deletion:**

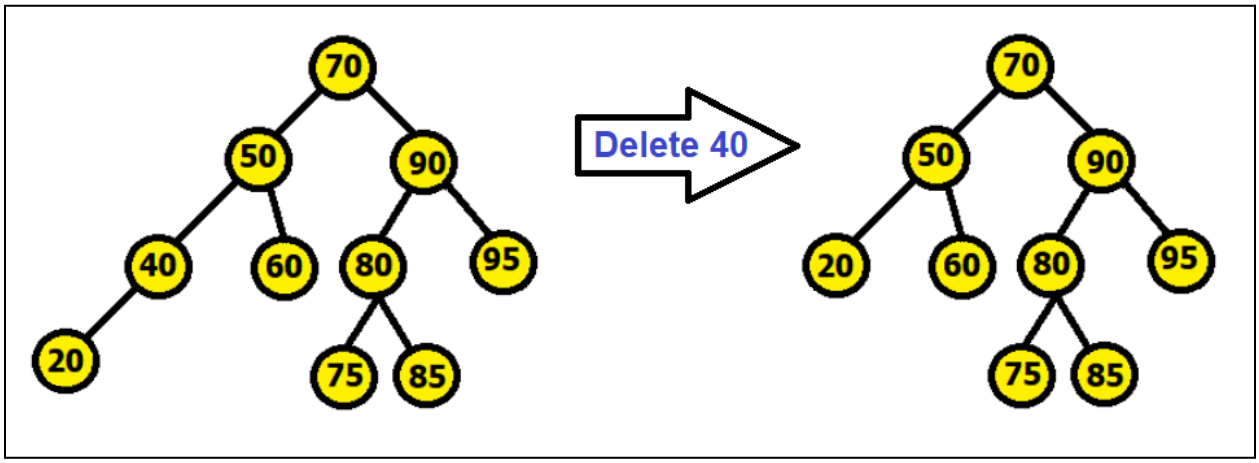
i) Case 1: No subtree or children:

You can simply delete the node without any additional actions.



ii) Case 2: One subtree or one child:

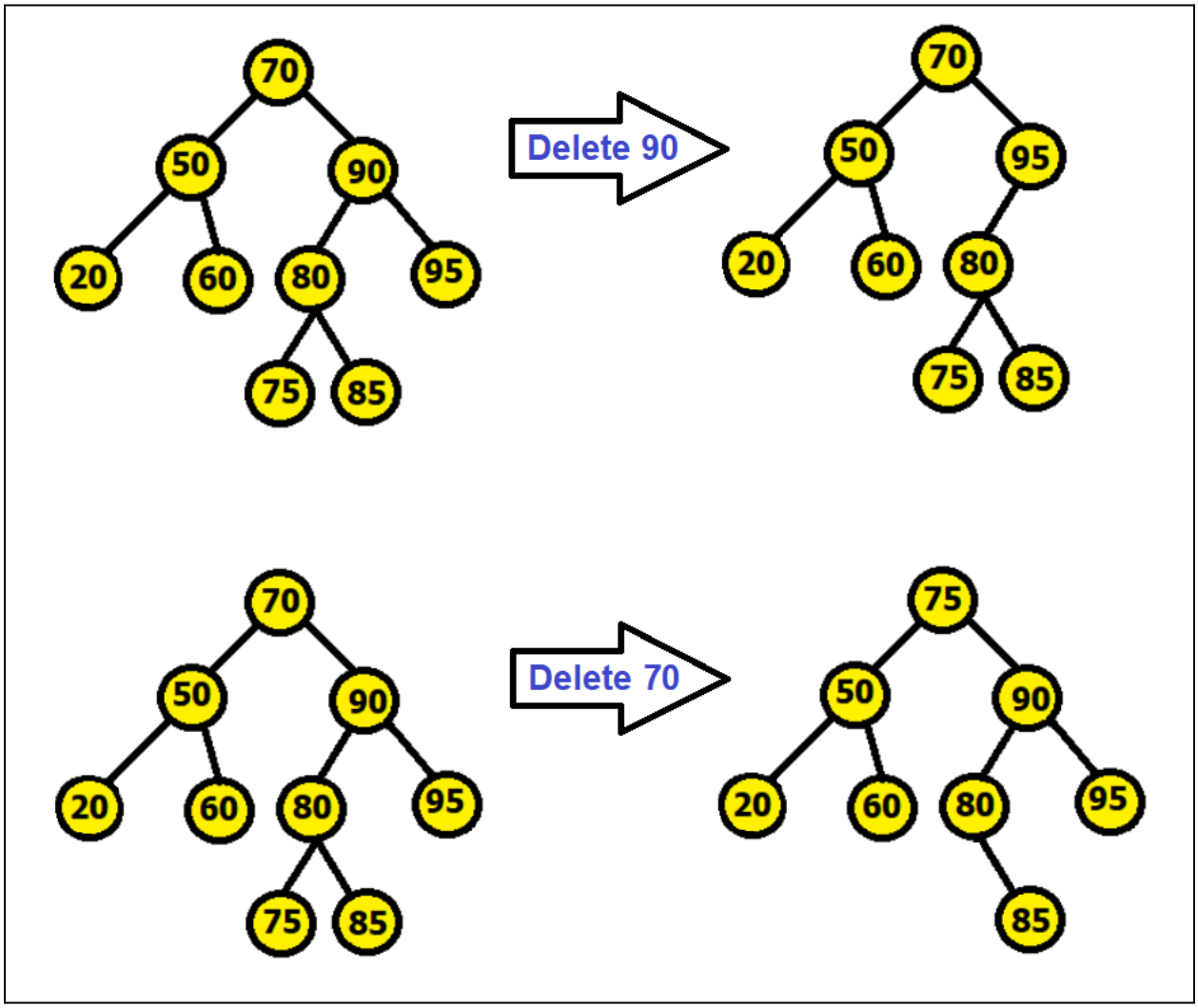
You have to ensure that after the node is deleted, its child is connected to the deleted node's parent.



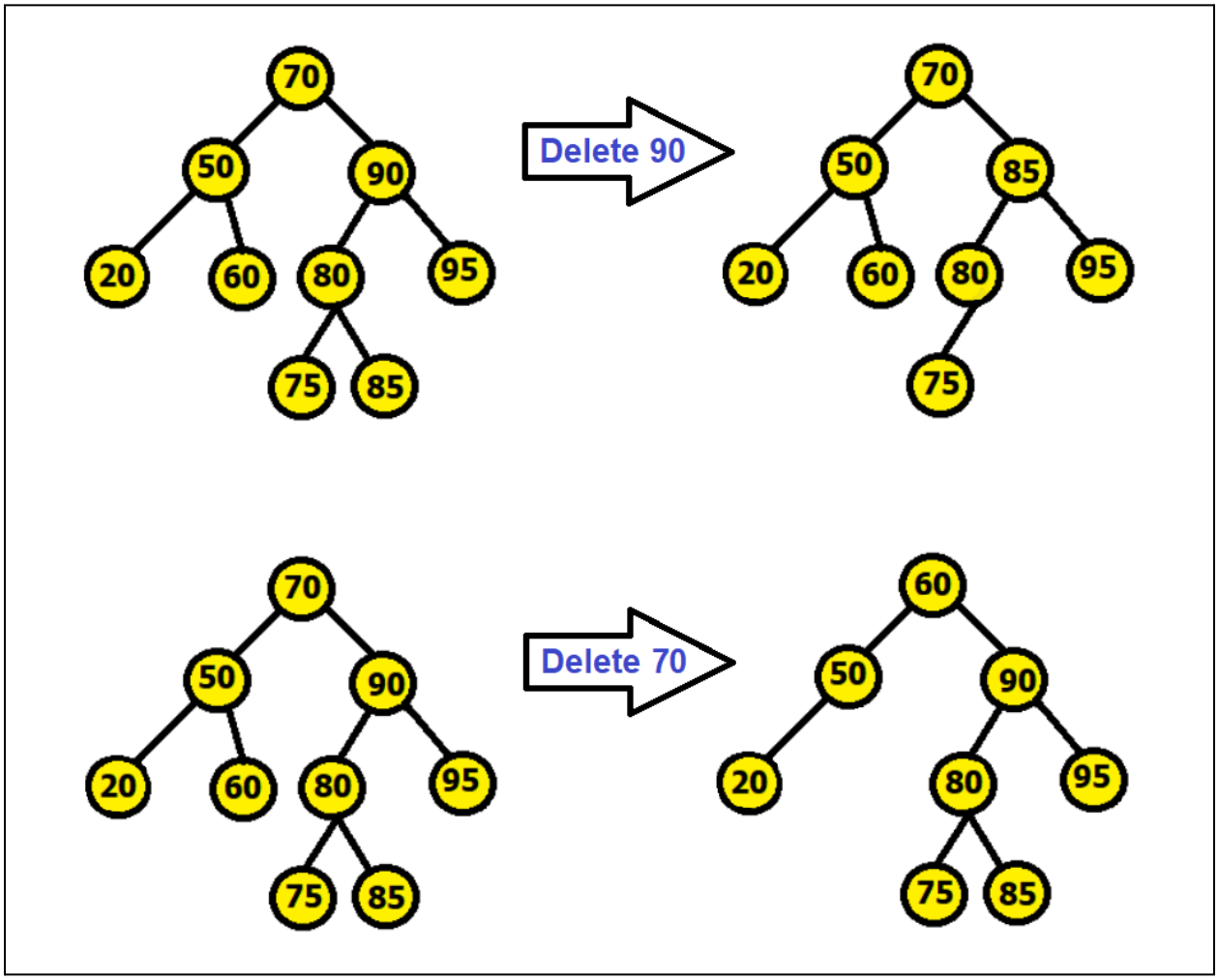
ii) Case 3: Two subtree or two children:

There are two ways a node can be deleted in Case 3.

Inorder Successor: Replace the deleted node with the leftmost node of its right subtree



Inorder Predecessor: Replace the deleted node with the rightmost node of its left subtree



- **BST Creation or Insertion:**

```
class Node:
        function constructor(elem):
                self.elem = elem
                self.left = null
                self.right = null
```

```
function addNode(root, value):
        if value < root.elem and root.left == null:
                n = new Node(value)
                root.left = n
        elif value > root.elem and root.right == null:
                n = new Node(value)
                root.right = n
        if value < root.elem and root.left != null:
                addNode(root.left, value)
        elif value > root.elem and root.right != null:
                addNode(root.right, value)

arr = new Array[70, 50, 40, 90, 20, 60, 20, 95, 99, 80, 85, 75]
root = new Node(arr[0])
for i = 1 to length(arr) - 1:
        addNode(root, arr[i])
```

- **Searching Items:**

```
function search(root, item):
        if root.elem == item:
                return root
        elif item < root.elem and root.left == null:
                return null
        elif item > root.elem and root.right == null:
                return null
        elif item < root.elem and root.left != null:
                return search(root.left, item)
        elif item > root.elem and root.right != null:
                return search(root.right, item)
```

- **Deletion (Inorder Successor):**

```
function leftMostNode(node):  #Finding leftmost node
        current = node
        while current.left != null:
                current = current.left
        return current

function deleteNode(root, key):
        if root == null:
                return null
        if key < root.elem:
                root.left = deleteNode(root.left, key)
        elif key > root.elem:
                root.right = deleteNode(root.right, key)
        else:
               # Case 1: No children
                if root.left == null and root.right == null:
                         return null

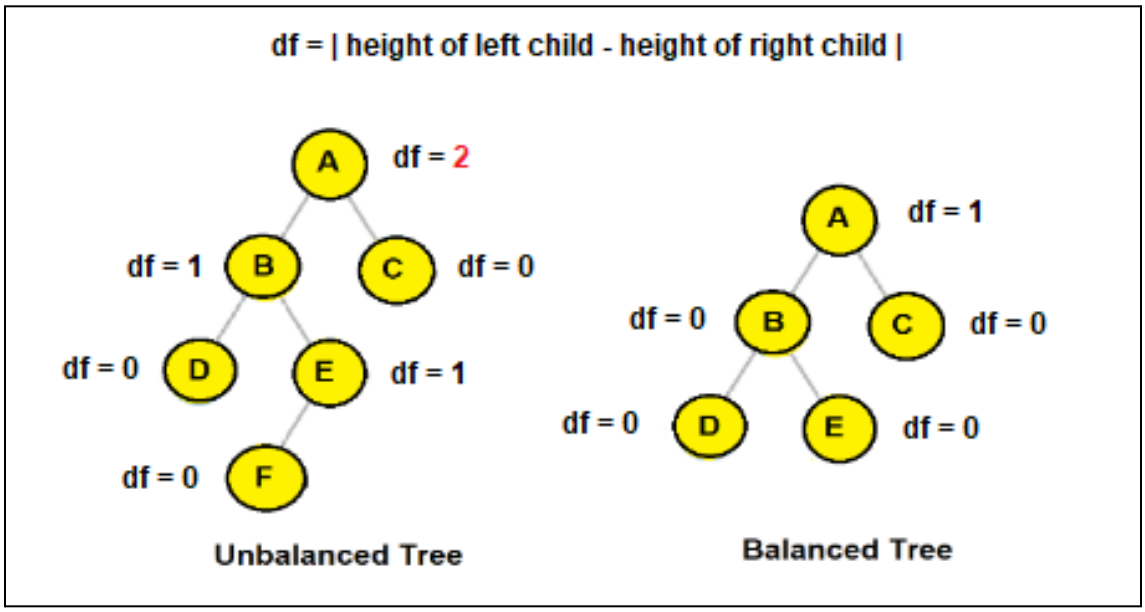
                # Case 2: One child
                elif root.left == null and root.right != null:  # Only right child
                        return root.right
                elif root.right == null and root.left != null::  # Only left child
                        return root.left

                # Case 3: Two children
                temp = leftMostNode(root.right)  #Leftmost of right subtree
                root.elem =  temp.elem  #Replace leftmost node with root
                root.right = deleteNode(root.right, temp.elem)
        return root
```

Balanced Binary Search Tree:

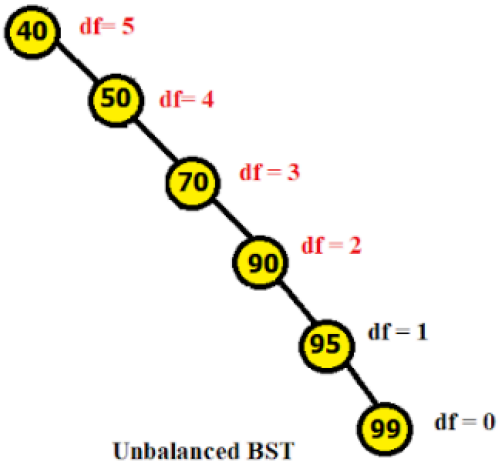
After Insertion and Deletion, a tree might be unbalanced. We have to make it balanced.

If the height difference between the left and right subtree of any node in the BST is more than one, it is an unbalanced BST. Otherwise, it is a balanced one. In a balanced BST, any searching operation can take up to O(log(n)) time complexity. In an unbalanced one, it may take up to O(n) time complexity, making BST usage obsolete. Therefore, we should only work with a balanced BST, and after conducting any operation on a BST, we must first check if it has become unbalanced or not. If it does become unbalanced, we have to balance it.



Conversion of an unbalanced BST into a balanced BST:

- Traverse the given BST in inorder and store the result in an array. This will give an ascending sorted order of all the values.
- Take the middlemost value in the (size/2) position in the array and make it the root.
- The left subtree of the root will be all the values from 0 to (size/2)-1 positions of the array. The right subtree of the root will be all the values from (size/2)+1 to (size-1) positions of the array.
- Again, choose the middlemost values from the left subtree and right subtree and connect these to the root. Keep on repeating the process until all the elements of the array have been taken.



Unbalanced BST

Inorder Traversal

40	50	70	90	95	99

array for original tree

size = 6
root position = (size//2) = 3
Pick 90 as the root

root



40	50	70

array for left subtree of node 90

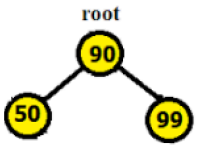
size = 3
node position = (size//2) = 1
Pick 50 as next left node

95	99

array for right subtree of node 90

size = 2
node position = (size//2) = 1
Pick 99 as next right node

root



40

array for left subtree of node 50

size = 1
node position = (size//2) = 0
Pick 40 as next left node

70

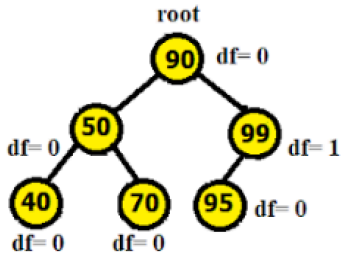
array for right subtree of node 50

size = 1
node position = (size//2) = 0
Pick 70 as next right node

95

array for left subtree of node 99

size = 1
node position = (size//2) = 0
Pick 95 as next left node



Balanced BST

- **Balancing BST:**

```
function pushTreeNodes(root, arr):
        if root == null:
                return
        pushTreeNodes(root.left, arr)
        arr.add(root)
        pushTreeNodes(root.right, arr)

function buildBalancedBST(arr, start, end):
        if start > end:
                return null
        mid = (start + end) / 2
        root = new Node(arr[mid])
        root.left = buildBalancedBST(arr, start, mid - 1)
        root.right = buildBalancedBST(arr, mid + 1, end)
        return root

arr = List()
pushTreeNodes(root, arr)
newRoot = buildBalancedBST(arr, 0, length(arr) - 1)
```